# CS4433 Software Engineering: Final Project Report

## Moderated Learning Management System (LMS)

Name: Kabelo Helen Khobotle

Student number: 202322667

Course: CS4433 Software Engineering, Department of Mathematics and Computer Science, National University of Lesotho

Development model: Waterfall with V-Model verification and validation

Git repository:https://github.com/KhobotleHelen/Software-Engineering-Moderate-LMS-.git

## Abstract

This report describes the full life cycle of a Moderated Learning Management System for a secondary school: planning, requirements, design, implementation, testing, quality assurance, deployment and maintenance. The system supports four roles (administrator, lecturer, Head of Department and student) and enforces the school's moderation rule that students see no mark until the Head of Department has approved and released it. It is written in C++17, stores data in SQLite, works offline, hashes passwords with Argon2id, and records every mark change in an audit log. The design uses a layered architecture with the Strategy, Repository and Factory patterns. The implementation is verified by 112 automated tests, with 98.3% line coverage of the business-logic layer, a clean static-analysis report, and a passing continuous-integration pipeline. The report also analyses the maintenance change request and discusses the legal, ethical and accessibility issues of the system.

*Table of contents: in Word, use References, then Table of Contents, after applying Heading 1 and Heading 2 styles to the section titles.*

# Section 2: Introduction

A secondary school needs a Learning Management System to manage courses, assignments and assessment. Users are students, lecturers, Heads of Department (HoDs) and a system administrator, and each may do only what their role allows. Each submission passes through a defined life cycle (submitted, late, marked, moderated, released), and every change to a mark must be recorded. Five stakeholders gave conflicting or incomplete statements: the Principal wants a simple system and HoD-checked marks; the lecturer wants quick results and the power to grant extensions; the HoD wants a late-work penalty and sight of every mark; the student representative wants resubmission and written feedback; and the IT officer needs an offline, secure and fast system for 800 students on old computers. Online payment, video conferencing, automatic grading, plagiarism detection and web or mobile interfaces are out of scope.

The project follows the Waterfall model over seven weeks, with V-Model test levels matched to each development artefact. The tools used are a UML tool for diagrams, a C++17 compiler, CMake, SQLite,

libsodium, Git with a hosted repository and GitHub Actions, GoogleTest, cppcheck, gcovr and lizard. The report follows the required order: project management plan (Section 3), requirements engineering (4), software design (5), implementation (6), verification and validation (7), configuration management and quality assurance (8), deployment (9), maintenance (10), professional, legal and ethical issues (11), conclusion (12) and references (13), with appendices.

# Section 3: Project Management Plan

## 3.1 Feasibility study

The feasibility study concluded that the LMS is possible to build and use as an academic project. The main conditions are testing on the school's own computers and confirming costs before any real deployment.

| Dimension | Finding | Condition |
|---|---|---|
| Technical | C++17 with SQLite is suitable; the system can run offline on one computer or a local network | Test with 800 students on the school's computers |
| Operational | Four roles with clear jobs; a moderation step is needed to resolve the Principal and lecturer conflict; menus must be simple for non-IT staff | Provide role-specific instructions and training |
| Economic | No extra services are needed; no project budget was given, so no costs are invented | The school must confirm budget, hardware and support staff before real use |
| Legal | The system holds personal data of children; the Data Protection Act 2012, Children's Protection and Welfare Act 2011 and Persons with Disability Equity Act 2021 apply | Re-check legal requirements before deployment |
| Schedule | Seven weeks is possible but tight, because phases depend on each other and implementation cannot start before Gate 2 | Watch the schedule and manage risks |

## 3.2 Work breakdown and schedule

| No. | Work package | Tasks |
|---|---|---|
| 1 | Project planning | Feasibility, work breakdown, schedule, risk management |
| 2 | Requirements engineering | Stakeholder analysis, conflict resolution, SRS, use cases |
| 3 | System design | Architecture, UML, patterns, database |

| No. | Work package | Tasks |
|---|---|---|
| 4 | Implementation | C++ development, SQLite, security, CI |
| 5 | Verification and validation | Unit, integration, system and acceptance testing |
| 6 | Configuration and quality assurance | Git, CI, static analysis, metrics, code review |
| 7 | Deployment | Deployment diagram, installation guide, manuals, v1.0 |
| 8 | Maintenance | CR1, impact analysis, v1.1, regression testing |
| 9 | Completion | Professional, legal and ethical issues, final report, viva |

### 3.3 Risk management

Six risks were identified (technical, project and people), each with likelihood, impact, exposure, mitigation and owner. The register was updated at the end of implementation; the original and updated versions are in Appendix B. The three highest risks (exposure 16) were old computers running the application slowly, an unreliable network, and requirement conflicts needing rework. The first two were handled by design (offline SQLite, performance test) and the third by resolving the conflicts in requirements engineering before the SRS was baselined.

## Section 4: Requirements Engineering

### 4.1 Stakeholders and conflict resolution

The stakeholders are the Principal, lecturers, Heads of Department, the student representative, the system administrator and IT officer, school management and the project developer. The statements conflicted or were incomplete in seven places, resolved as follows.

| Conflict | Resolution |
|---|---|
| 1. When students see marks (Principal against lecturer) | Lecturer marks and writes feedback; the HoD approves; students see marks only after release |
| 2. Extensions (rules missing) | Lecturer records an individual extension; the new deadline is used to decide lateness; school to confirm whether proof is needed |
| 3. Late work (HoD penalty against student wish for no rejection) | Late work is accepted for up to three days with a 10% per day penalty; after that it is accepted but receives zero; a part-day counts as a full day |
| 4. Resubmission (limit and version unclear) | Students may resubmit before the deadline; the |

| Conflict | Resolution |
|---|---|
| | latest valid version is marked |
| 5. Written feedback (not always required) | Feedback is saved with the mark and shown only after release; implementation requires feedback for every mark |
| 6. Offline and fast | Local SQLite database, no internet needed, passwords as salted hashes, tested with 800 students |
| 7. Easy to use | Simple role-specific menus and clear messages |

## 4.2 Software requirements specification

The SRS follows ISO/IEC/IEEE 29148 and has 14 functional and 8 non-functional requirements, each uniquely identified, prioritised with MoSCoW and verifiable.

| ID | Functional requirement | Priority |
|---|---|---|
| FR-01 | Users log in with their account details | Must |
| FR-02 | Each user is limited to what their role allows | Must |
| FR-03 | The administrator creates and manages user accounts | Must |
| FR-04 | The administrator manages courses, departments and enrolments | Must |
| FR-05 | Lecturers create assignments with title, instructions, deadline and total mark | Must |
| FR-06 | Students see the assignments of their courses | Must |
| FR-07 | Students submit work | Must |
| FR-08 | Students fix and resubmit work before the deadline | Must |
| FR-09 | Lecturers record an extension for a student | Must |
| FR-10 | The system works out late penalties from the correct deadline | Must |
| FR-11 | Lecturers record marks and written feedback | Must |
| FR-12 | The HoD reviews marks and approves release before students see them | Must |
| FR-13 | Every mark change is recorded: who, when, old mark, new mark, reason | Must |
| FR-14 | Users see the information they need for their work | Should |

| ID | Non-functional requirement (measurable) | Priority |
|---|---|---|
| NFR-01 | Login takes 2 seconds or less with 800 students stored | Should |
| NFR-02 | A normal database action takes 2 seconds or less with 800 students | Should |
| NFR-03 | Main functions work with no internet | Must |
| NFR-04 | Passwords are stored only as salted hashes from a trusted library | Must |
| NFR-05 | A user cannot see or change what their role does not allow | Must |
| NFR-06 | Bad input or database problems give a clear message and no crash | Must |
| NFR-07 | Clear menus for each role's main tasks, checked with one test user per role | Should |
| NFR-08 | Every mark change has a full audit record | Must |

### 4.3 Use case model

The use case diagram covers the four actors. Every action includes Log in; Submit assignment includes the late-penalty check; and Resubmit extends Submit assignment. Three use cases are fully specified: Submit assignment (student), Create assignment (lecturer) and Review and release marks (HoD), each with preconditions, main flow, alternative and exception flows, postconditions and business rules.

## Section 5: Software Design Document

### 5.1 Architecture

A layered architecture was chosen: presentation layer, application and business-logic layer, data-access layer and SQLite database. Each layer has one job, which makes the code easier to maintain and test, lets access checks happen before protected operations, keeps all database access in one place, and allows offline operation. A single unstructured program mixing menus, rules and SQL was considered and rejected because a change in one place could break unrelated functions and database access could not follow consistent security rules.

### 5.2 Class design and design patterns

The class diagram has an abstract `User` with `Student`, `Lecturer`, `HeadOfDepartment` and `Administrator` subclasses; domain classes `Assignment`, `Submission`, `Extension`, `Course`, `Department`, `Enrolment`, `MarkAudit` and `Moderation`; services for submission and assessment; and repository interfaces with SQLite implementations. The Strategy pattern separates the late-penalty rule so a policy change needs no change to the services; the Repository pattern keeps SQL out of business classes and supports testing; a Factory pattern (added during implementation) creates the right user subclass. The design follows the SOLID principles: single responsibility per service, open/closed late-penalty strategies, substitutable user subclasses, small repository interfaces, and services depending on interfaces rather than SQLite.

### 5.3 Behavioural models

The submission life cycle is a state machine with the states Submitted, Late, Marked, Moderated and Released. A student submission becomes Submitted or Late depending on the deadline; the lecturer's marking moves it to Marked; HoD approval moves it to Moderated and release moves it to Released; a correction request returns it to the lecturer while it stays Marked. The swimlane activity diagram follows the process from submission to released mark across student, lecturer, HoD and system, and sequence diagrams show Submit assignment (with late and resubmission paths) and Moderate and release marks.

### 5.4 Database design

The SQLite database has ten tables: User, Department, Course, CourseLecturer, Enrolment, Assignment, Extension, Submission, MarkAudit and Moderation, normalised to Third Normal Form. Keys, foreign keys and checks are defined in the SQL schema, and foreign-key enforcement is switched on for every connection. Each table maps to a class; the one table `User` maps to the abstract class and its four subclasses through a role column.

### 5.5 Design changes found during implementation

Implementation exposed gaps in the Phase 3 design, which were corrected. A `head_id` column was added to `Department` so an HoD can be tied to a department, because the design required HoDs to be checked against their department but had no way to record it. Two database triggers now enforce that a mark never exceeds the assignment maximum, and seven indexes support the 2-second targets. A `UserFactory` was added, and the component services were realised as `AuthService`, `AccessControl`, `UserService`, `AcademicService`, `AssignmentService`, `SubmissionService` and `AssessmentService`. These changes should appear in the final versions of the class diagram, ERD, data dictionary and SQL schema.

## Section 6: Implementation

*Moderated Learning Management System, CS4433 Final Project, Phase 4*

### 6.1 Overview and technology choices

The system is a C++17 console application that stores its data in a local SQLite file, so every main function works offline (NFR-03). It builds with CMake and is organised in the same layers chosen in the Phase 3 design: a presentation layer (`ui`), a business-logic layer (`services`, `strategy`, `domain`, `common`, `security`), a data-access layer (`data`) and the SQLite database. A console interface was chosen over a desktop GUI because the school computers are old, the teachers are not IT experts and the Principal asked for simplicity; short numbered menus per role are easier to learn, easier to test automatically and have no graphics-library dependency.

The code base is about 2,900 non-comment lines in `src` plus a test suite of 112 automated tests. It compiles without warnings under `-Wall -Wextra -Wpedantic -Wshadow -Wconversion` and with warnings treated as errors in the CI build.

| Layer | Folder | Main contents |
|---|---|---|
| Presentation | `src/ui` | `ConsoleApp`: role menus, input reading, message display. No SQL and no business rules. |
| Business logic | `src/services`, `src/strategy`, `src/domain`, `src/common`, `src/security` | Services, access control, late-penalty strategy, user classes, validation, password hashing. |
| Data access | `src/data` | `Database`/`Statement` wrappers, repository interfaces and their SQLite implementations, SQL schema. |
| Composition | `src/services/AppContext.h`, `src/main.cpp` | Creates the concrete objects and connects them (the only place that names SQLite classes). |

## 6.2 Mapping from design to code

Every Phase 3 component was realised as a class group, so the code follows the design:

| Phase 3 component or class | Implementation |
|---|---|
| Authentication and access control | `AuthService`, `AccessControl`, `Permission` enum |
| Course and user management service | `UserService`, `AcademicService` |
| Assignment service | `AssignmentService`, `DeadlineResolver` |
| Submission service | `SubmissionService` |
| Assessment and moderation service | `AssessmentService` |
| Abstract `User`, `Student`, `Lecturer`, `HeadOfDepartment`, `Administrator` | `domain/User.h`: abstract base with pure virtual `role()` and `permissions()`; four `final` subclasses |
| `ILatePenaltyStrategy`, `StandardLatePenalty` | `strategy/LatePenalty.h` |
| Repository interfaces and SQLite implementations | `data/Repositories.h` (nine interfaces) and `data/SqliteRepositories.cpp` |
| Assignment, Submission, Extension, Course, Department, Enrolment, MarkAudit, Moderation | plain structs in `domain/Entities.h`; each maps to one table |

Three things were added or changed while implementing, and are recorded as design changes (see the deviations list in the repository): a `UserFactory` (third design pattern); a `head_id` column on `Department` so an HoD can be tied to a department (the Phase 3 design required this check but had no column for it); and database triggers and indexes. These changes should be reflected in the final versions of the Phase 3 diagrams.

## 6.3 Design patterns and object-oriented features

**Strategy (late penalty).** `SubmissionService` and `AssessmentService` receive an `ILatePenaltyStrategy` by reference. `StandardLatePenalty` implements the HoD's rule: 10% of the *maximum* mark for each day late for up to three days, and zero after that. A part of a day counts as a full day, and work submitted exactly at the deadline is on time. A unit test creates a different strategy (50% per day, one day) and shows that the services need no change, which demonstrates the Open/Closed Principle.

**Repository/DAO.** Services depend only on interfaces such as `ISubmissionRepository`. All SQL is inside `SqliteRepositories.cpp`, always through the `Statement` class with bound parameters. A test supplies a failing audit repository to prove the services work against the interface, not against SQLite (Dependency Inversion).

**Factory.** `UserFactory::create(role, ...)` is the single place that turns a stored role into the correct `User` subclass.

**Inheritance and polymorphism.** `User` is abstract. Each role overrides `permissions()`, and `User::can()` is used by `AccessControl` without knowing the concrete role (Liskov Substitution). The UI dispatches to a role menu through `User::role()`.

**Other SOLID decisions.** Each service has one responsibility (SRP); repository interfaces are small and separate (ISP); the clock, password hasher and transaction runner are injected so tests can control time, speed and failures.

## 6.4 Implementing the business rules

| Rule | Where and how |
|---|---|
| Role-based access (FR-02) | Each role has a fixed permission set. Every service method first calls `AccessControl::require`, then a scoped check: a lecturer must teach the course, a student must be enrolled, an HoD must head the course's department. The UI also hides menu items, but security does not depend on the UI. |
| Extensions (FR-09) | One extension per student per assignment, stored in `Extension`; it must be later than the original deadline and have a reason. `DeadlineResolver` returns the later of the original and extended deadline and is used for both submission and marking. |
| Resubmission (FR-08) | Allowed only before the effective deadline and only while the work is still Submitted or Late (not yet marked). The same submission row is updated; the latest version is the one marked. |
| Late work (FR-10) | Never rejected. It is saved with status Late and the student is told the number of days late. After three days it |

| Rule | Where and how |
|---|---|
| | is still accepted but will receive zero, so nothing is silently refused. |
| Marking (FR-11) | The lecturer enters the raw mark and written feedback (feedback is mandatory). The system applies the penalty, stores the final mark and shows the lecturer the breakdown. A mark cannot be negative or above the assignment maximum. |
| Moderation and release (FR-12) | State machine Submitted/Late to Marked to Moderated to Released. The HoD can approve or require a correction (comments mandatory); release is refused unless the latest moderation outcome is Approved. A lecturer can no longer change a mark once it is Moderated. |
| Hidden marks | `resultsForStudent` removes the mark and feedback from every submission that is not Released, so the rule is enforced in the service and cannot be bypassed by a UI bug. |
| Audit log (FR-13, NFR-08) | Every new or changed mark writes a `MarkAudit` row (who, when, old, new, reason) in the **same database transaction** as the mark update. A change to an existing mark requires a reason. A test injects a failing audit write and shows the mark update is rolled back. |

## 6.5 Persistence

SQLite is accessed only through the data-access layer. The schema is the Phase 3 schema plus the documented additions. Foreign-key enforcement is switched on for every connection. `Database::run()` wraps work in a transaction, joins an outer transaction when nested, and rolls back on any exception. Because the Phase 3 schema allows a mark above the maximum unless the application checks it, the check is done twice: in `AssessmentService` and in two SQLite triggers. Seven indexes support the queries used by the role menus so that the 2-second targets (NFR-01, NFR-02) are met with 800 students.

## 6.6 Security

**Password storage (NFR-04).** Passwords are hashed with **Argon2id through libsodium** (`crypto_pwhash`). Each password gets its own random 16-byte salt from the library's secure random generator; the 32-byte hash and the salt are stored as hex in the `password_hash` and `salt` columns. Hashes are compared with `sodium_memcmp`, which takes constant time. The *interactive* cost parameters (2 passes, 64 MiB of memory) were used; hashing takes about 55 ms on the development machine, so a login stays far below the 2-second limit, while an attacker who steals the database must spend that memory and time on every guess.

Justification of the choice: libsodium is a widely reviewed library, so no cryptography was written by hand. Argon2id won the Password Hashing Competition and is the algorithm currently recommended by OWASP. It is memory-hard, which makes attacks on graphics cards expensive; the alternatives are weaker for this threat (plain SHA-256 is far too fast and unsalted by default; PBKDF2 is not memory-hard; bcrypt is acceptable but limits password length to 72 bytes). libsodium is a small local library, needs no network, and is available for Windows, Linux and macOS, which suits the offline requirement. The hasher sits behind the `IPasswordHasher` interface, so the algorithm can be replaced later. A test confirms that equal passwords produce different salts and hashes and that the stored value never contains the password.

**Other measures.** Login failures give the same message for an unknown user and a wrong password and take similar time (a dummy hash is verified for unknown users), which prevents user enumeration. Passwords must be 8 to 128 characters. All SQL uses bound parameters, and tests show that injection strings such as `' OR '1'='1` find nothing. Deactivated accounts cannot log in. Password entry hides the characters typed on a terminal. Internal error detail goes to `lms_error.log`, never to the screen.

## 6.7 Error handling and validation

A single exception hierarchy (`ValidationError`, `AuthenticationError`, `AuthorizationError`, `NotFoundError`, `BusinessRuleError`, `DatabaseError`, all deriving from `LmsError`) is used by all layers. Every menu action runs inside one guard in the UI that converts any exception into a short message, so invalid input or a database failure returns the user to the menu instead of crashing. Validation covers usernames, passwords, text lengths, marks, maximum marks, dates (including leap years and impossible dates), deadlines in the past and menu input. A system test deliberately drops a table and shows the user sees only a generic message while the program keeps running.

## 6.8 Build and coding standard

The project builds with CMake (`lms_core` library, `lms` application, `lms_seed` data generator, `lms_tests`). The coding standard (naming, layering rules, RAII, no raw `new`/`delete`, const-correctness, error handling, security rules, function size) is documented in `docs/CODING_STANDARD.md` and enforced by `.clang-format`, strict compiler warnings and cppcheck. The build guide is `docs/BUILD.md`.

## 6.9 Implementation results (evidence for Phases 5 and 6)

| Measure | Result |
|---|---|
| Automated tests | 112 tests in 19 suites, all passing (unit, integration with real SQLite, scripted console system tests, performance test) |
| Line coverage, business-logic layer (services, strategy, domain, common, security) | **98.3%** (711 of 723 lines) |
| Line coverage, all of `src` except `main.cpp` | 92.5% (1,550 of 1,676 lines) |
| Static analysis (cppcheck, all checks enabled) | 12 style/performance findings fixed; 0 remaining |

| Measure | Result |
|---|---|
| Cyclomatic complexity (lizard) | average 2.5; highest 14 (a menu dispatcher), below the limit of 15 set in the coding standard |
| Performance, 800 students and 2,400 marks on the development machine | login with real Argon2id cost and all list queries well under 2 s (the complete test runs in about 0.5 s) |

The performance figures come from the development computer, not the school's machines. The same test (`ctest -R Performance`) must be repeated on the school's computer during acceptance testing before NFR-01 and NFR-02 are approved.

### 6.10 Limitations

Submissions store the path to the file rather than a copy, so on a network the path must point to a shared folder. There is no lock-out after repeated failed logins and no session time-out. The database file is not encrypted and relies on operating-system permissions and backups. Approved marks cannot be re-opened; a corrected mark after approval would need a new use case. Only the latest version of a resubmission is kept, as noted in the Phase 3 schema notes. These limits are acceptable for the project scope and are candidates for future change requests.

## Section 7: Verification and Validation

### 7.1 Test strategy (V-Model)

Testing follows the V-Model: each development artefact has a matching test level, and the tests were written from the artefact, not from the code.

| V-Model level | Artefact verified | Test approach | Entry criteria | Exit criteria |
|---|---|---|---|---|
| Unit | Class design (Phase 3 class diagram) | GoogleTest on `StandardLatePenalty`, validation, `DateTime`, hasher, `User` roles | Code compiles with no warnings | All unit tests pass; business-logic coverage at least 70% |
| Integration | Architecture and database design | Services against a real in-memory SQLite database; repositories, transactions, constraints, triggers | Unit tests pass | All integration tests pass; schema constraints demonstrated |
| System | SRS and use case specifications | Scripted console sessions that drive the real menus, including exception flows | Integration tests pass | Every Must requirement exercised; no open high-severity defect |
| Acceptance | Stakeholder statements and | Principal, lecturer, HoD, student and IT-officer | System tests | Rules agreed with stakeholders hold; |

| V-Model level | Artefact verified | Test approach | Entry criteria | Exit criteria |
|---|---|---|---|---|
| | NFRs | rules replayed as tests; performance test with 800 students | pass | NFR-01 and NFR-02 repeated on the school computer |

Equivalence partitioning and boundary value analysis were applied to the rules the stakeholders care most about:

| Input | Partitions | Boundary values tested |
|---|---|---|
| Submission time against deadline | on time; 1 to 3 days late; more than 3 days late | deadline minus 1 s, deadline, deadline plus 1 s, plus 1 day, plus 3 days, plus 3 days and 1 s |
| Mark after penalty (maximum 100, raw 80) | 0, 1, 2, 3, 4 or more days late | 80, 70, 60, 50, 0; also a raw mark smaller than the penalty (result 0, never negative) |
| Mark entered | valid; below 0; above maximum | -0.01, 0, 100, 100.01 |
| Password length | too short; valid; too long | 7, 8, 128, 129 characters |
| Assignment deadline | past; future | now, now minus 1 s, now plus 1 s |
| Extension date | not later than original; later | original deadline, original plus 1 s |

## 7.2 Automated tests and coverage

The suite has **112 tests in 19 suites**, built into one executable (`lms_tests`) and run by `ctest` and by the CI pipeline. The coverage figures below were produced with gcov and gcovr.

| Measure | Result |
|---|---|
| Tests run / passed | 112 / 112 |
| Line coverage, business-logic layer (services, strategy, domain, common, security) | **98.3%** (711 of 723 lines); target was 70% |
| Line coverage, all of `src` except `main.cpp` | 92.5% (1,550 of 1,676 lines) |
| Function coverage, all of `src` | 97.7% |

Branch coverage is lower (about 55%) because many branches are defensive checks for cases that cannot be reached through the public interfaces, such as an impossible role value read from the database. The lines not covered are in the same category (unknown-enum fallbacks and a library-failure path in the hasher).

## 7.3 System and acceptance tests

System tests drive the real `ConsoleApp` with scripted keystrokes against a real SQLite database. They include the main flow of each use case and its exception flows.

| Test | Derived from | What it shows |
|---|---|---|
| First-run setup and login | FR-01, FR-03 | A password mismatch is refused and retried; the administrator is created and can log in |
| School set-up through the menus | UC: manage users and academic data (FR-03, FR-04) | Users, department, head, course, lecturer and enrolment created; invalid role and duplicate enrolment rejected with messages |
| End to end: submit, mark, moderate, release | UC1 Submit assignment, UC3 Review and release marks, Principal's rule | The student's marks stay hidden after marking and after approval until the HoD releases them, then appear with feedback |
| Late submission and penalty | HoD late-work rule | Two days late: message shown to the student; lecturer sees penalty 20 and final mark 70 |
| Extension flow | Lecturer statement | Extension recorded; student sees the new deadline; submission after the original deadline counts as on time |
| Exception flows | NFR-06 | Invalid menu number, non-numeric input, mark above maximum, empty feedback, bad date and unknown submission all give a friendly message and the program continues |
| Role access | FR-02, NFR-05 | Student and lecturer menus do not offer other roles' actions; a lecturer cannot mark work of another department |
| Database failure | NFR-06 | With the `Submission` table dropped the user sees a generic message and the program keeps running; no internal text is shown |

NFR-07 (usability) is the one requirement still to be verified: it needs one test user from each role to do a main task, and has not yet been run.

## 7.4 Defect log

The tests found no functional defects in the delivered business logic. The defects recorded during the build are:

| ID | Description | Severity | Cause | Fix | Retest |
|---|---|---|---|---|---|
| D1 | CMake configuration | Medium | `if(...)` and `endif()` | Put each | Build |

| ID | Description | Severity | Cause | Fix | Retest |
|---|---|---|---|---|---|
| | failed with a parse error | (blocks build) | written on one line | on its own line | succeeds |
| D2 | Test file failed to compile (`nan`, `INFINITY` undeclared) | Low | Missing `<cmath>` include | Added include | Build succeeds |
| D3 | One test failed: changing a mark with no reason was expected to succeed | Low (test defect) | The test contradicted the rule that a changed mark needs a reason; the code was correct | Corrected the test and added the explicit reason case | 112 of 112 pass |
| D4 | cppcheck reported 12 style and performance findings | Low | Parameter names differing from declarations, raw loops, a function that could be static or const, one unused private function | Fixed all 12 | 0 findings; 112 of 112 pass |

# Section 8: Configuration Management and Quality Assurance

## 8.1 Configuration management

The source is kept in Git. The branching strategy is: `main` always builds and passes CI; work is done on short-lived `feature/` and `fix/` branches that are merged by pull request when CI is green; commit messages are short, imperative and describe one change; releases are annotated tags (`v1.0`, `v1.1`). The strategy is documented in `docs/GIT_WORKFLOW.md`.

A GitHub Actions workflow (`.github/workflows/ci.yml`) runs on every push and pull request. It installs the dependencies, builds with coverage and warnings as errors, runs all tests, fails if business-logic line coverage drops below 70%, and runs cppcheck with a failing exit code on findings. Evidence of a passing run (screenshot of the Actions tab) is attached in the appendix.

## 8.2 Quality assurance

**Static analysis.** cppcheck with all checks enabled reported 12 findings, all style or performance. All were fixed and the re-run reports none (files `cppcheck_before.txt` and `cppcheck_after.txt`).

**Metrics (lizard).** 2,887 lines of code in `src` across 236 functions; average function length 7.9 lines; average cyclomatic complexity 2.5. The most complex functions are menu dispatchers in the user interface (complexity up to 14), `DateTime::fromParts` (13), `AssessmentService::recordMark` (10) and `SubmissionService::submit` (9). None exceeds the limit of 15 set in the coding standard.

**Self code review.** The code was reviewed against the checklist below and the following changes were made.

| Checklist item | Finding | Change made |
|---|---|---|
| Layering: no SQL in services or UI, no business rules in UI | Satisfied | None |
| All SQL parameterised | Satisfied | None |
| Passwords never stored or logged in plain text | Satisfied | None |
| Permissions checked in the service, not only in the UI | Satisfied | None |
| Const-correctness, no raw `new`/`delete` | Satisfied | `AccessControl::require` made `static` (it uses no member data) |
| Naming consistent with the standard | Parameter names differed between declaration and definition in five places | Renamed to match |
| No dead code | One unused private function in `ConsoleApp` | Removed |
| Prefer standard algorithms to raw loops | Two list-building loops and one validation loop | Replaced with `std::transform` and `std::all_of` |
| Formatting | Inconsistent line breaks | Applied `.clang-format` to all files |

# Section 9: Deployment

## 9.1 Deployment view

The system is deployed on one school computer, or on one computer that other users reach over the local network. The deployment diagram has these nodes: the school computer (node) hosting the executable `lms` and the SQLite database file `lms.db` (artefacts) with the error log `lms_error.log`; an optional shared network folder where students and lecturers store the files named in submissions; and the libraries SQLite and libsodium linked into the executable. No node needs internet access.

## 9.2 Installation guide for a school technician

1. Install a C++ build environment, CMake, SQLite and libsodium (commands for Ubuntu, macOS and Windows are in `docs/BUILD.md`).

2. Copy the project folder to the school computer.
3. Build: `cmake -B build -DCMAKE_BUILD_TYPE=Release` then `cmake --build build -j`.
4. Verify: `ctest --test-dir build --output-on-failure`; all 112 tests must pass.
5. Choose a folder for the database that is backed up regularly, and start the program with `./build/lms /path/to/lms.db`.
6. On first start, create the administrator account (password of at least 8 characters).
7. As administrator, create departments, courses, lecturers, HoDs and students, assign each HoD to a department, and enrol students.
8. Run `ctest -R Performance` on this computer to confirm the 2-second targets.

### 9.3 User manual summary

| Role | Menu options |
|---|---|
| Student | My courses; My assignments; Submit or resubmit work; My released marks and feedback; Change my password |
| Lecturer | My courses; Create an assignment; List assignments of a course; Grant a student an extension; View submissions; Mark a submission; View mark history; Change my password |
| Head of Department | Marks waiting for review or release; Review a submission (approve or request correction); Release approved marks; View mark history; My department's courses; Change my password |
| Administrator | Create a user account; List users; Activate or deactivate an account; Reset a password; Create a department; Set a department's Head; Create a course; Assign a lecturer; Enrol a student; List departments and courses; Change my password |

Each menu is chosen by typing its number; 0 logs out. Dates are typed as `YYYY-MM-DD HH:MM`. Full step-by-step role manuals go in the appendix.

### 9.4 Release notes, v1.0

First release. All Must requirements (FR-01 to FR-13) and the non-functional requirements NFR-03 to NFR-06 and NFR-08 are implemented and tested. Known limitations: submissions store a file path rather than a copy of the file; there is no account lock-out or session time-out; the database file is not encrypted; approved marks cannot be re-opened. The release is tagged `v1.0` in Git.

## Section 10: Maintenance (Change Request CR1)

**CR1:** a lecturer should be able to extend the deadline for a whole class, and extensions should be shown on the student's assignment list.

### 10.1 Classification and impact analysis

This is **perfective maintenance**: the software is not wrong and the environment has not changed; users want a new capability and a better display. (The second half, showing extensions, is also a small usability improvement.)

| Area | Affected items |
|---|---|
| Requirements | FR-09 (extension) extended by a new requirement for class-wide extensions; FR-06 (view assignments) changed to show extensions; NFR-07 (usability) |
| Use cases and diagrams | Use case "Grant extension" (new alternative: whole class); use case diagram; sequence diagram of submission (no change to flow); class diagram |
| Classes | `AssignmentService` (new `grantClassExtension`); `StudentAssignmentView` (add effective deadline and extension flag); `ConsoleApp` (lecturer menu, student assignment list); `IEnrolmentRepository::studentIdsOfCourse` (reused, no change) |
| Tables | No new table or column: a class extension is stored as one `Extension` row per enrolled student, written in one transaction, so `UNIQUE(assignment_id, student_id)` and the existing deadline logic keep working |
| Tests | Existing: `Extensions.*`, `Assignments.StudentSeesOnlyAssignments...`, `SystemTest.ExtensionFlowThroughUi`; new: class extension covers all enrolled students, none outside the course, rolls back on failure, extension shown on the list |
| Risk | Late-penalty and submission logic read the effective deadline through `DeadlineResolver`, so they need no change; regression tests must confirm this |

### 10.2 Implementation and regression results

*Not yet completed.* The v1.1 implementation, the updated design documents and the regression test run (existing 112 tests plus the new CR1 tests) still have to be done and tagged `v1.1`. This section must be completed with those results before submission.

## Section 11: Professional, Legal and Ethical Issues

**Data protection.** The system stores names of users, enrolments, submissions, marks and feedback of secondary-school learners, who are children. The Lesotho Data Protection Act, 2012 and the Children's

Protection and Welfare Act, 2011, identified in the feasibility study, require that personal data is collected only for a purpose, kept secure and handled with care for children. The design collects only what teaching and assessment need (a username and role, no addresses, ID numbers or contact details), limits each role to the records it needs (Phase 3 access control, tested in the system tests), and keeps the data on the school's own computer with no internet transfer. The legal requirements should be checked again with the school before real use, and the school must decide how long records are kept and how learners can ask for corrections.

**Security of assessment records.** Passwords are stored as salted Argon2id hashes; all database access uses parameterised queries; every mark change is recorded with who, when, old value, new value and reason in the same transaction as the change; unreleased marks are never returned to students; and marks cannot be changed after the HoD approves them. Remaining risks are the unencrypted database file, which must be protected by operating-system permissions and backups, and the lack of account lock-out and session time-out.

**Accessibility.** The console interface uses plain text, numbered menus, no colour and no time limits, so it works with screen readers and with keyboard-only use, and it needs no graphics hardware on the old school computers. It does not offer large-text or high-contrast settings of its own, relying on the terminal's settings, and it has not been tested with learners with disabilities. A later release should be reviewed against the Persons with Disability Equity Act, 2021 together with a disability-support officer.

**Fairness of the late-penalty rules.** The rules are applied by the system the same way to every student, so there is no human inconsistency, and the penalty is recorded in the audit log for review. They are not entirely neutral, though. Counting a part-day as a whole day can penalise a student who is seconds late; taking 10% of the *maximum* mark rather than of the mark earned hits weaker students proportionally harder; and giving zero after three days is severe for a student with a genuine problem. The design reduces the harm with lecturer-granted extensions (with a recorded reason), by accepting late work instead of rejecting it, and by making the lecturer see the penalty before the mark goes to moderation. The school should confirm the rule with stakeholders and consider showing students how a penalty was calculated.

**Professional responsibility.** The work follows the ACM Code of Ethics (avoid harm, be honest about limitations, respect privacy): the known limitations are stated in the report, tests are reported with their real results, and security and privacy were treated as requirements from the start.

## Section 12: Conclusion

The project took the Moderated LMS from planning through requirements and design to a working, tested C++17 application. The implementation follows the layered design, uses the Strategy, Repository and Factory patterns, stores data in SQLite through a data-access layer, protects passwords with Argon2id, and records every mark change in an audit log. 112 automated tests pass, business-logic line coverage is 98.3%, and static analysis is clean. The main lesson was that implementation exposes gaps that design reviews miss: the missing link between a Head of Department and a department, and the open rules about part-day lateness and resubmission, only became clear when the rules had to be executed. Remaining work is the NFR-07 usability test, performance confirmation on the school's own computer, the CR1 change (v1.1) and the release tagging. Future improvements are account lock-out,

keeping all versions of a resubmission, re-opening approved marks with a recorded reason, and a graphical interface.

# Section 13: References


ACM (2018) *ACM Code of Ethics and Professional Conduct*. New York: Association for Computing Machinery.

Biryukov, A., Dinu, D. and Khovratovich, D. (2016) 'Argon2: new generation of memory-hard functions for password hashing and other applications', *IEEE European Symposium on Security and Privacy*, pp. 292-302.

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley.

ISO/IEC/IEEE (2018) *ISO/IEC/IEEE 29148:2018 Systems and software engineering: Life cycle processes: Requirements engineering*. Geneva: ISO.

Kingdom of Lesotho (2012) *Data Protection Act, 2012*. Maseru: Government of Lesotho.

Kingdom of Lesotho (2011) *Children's Protection and Welfare Act, 2011*. Maseru: Government of Lesotho.

Kingdom of Lesotho (2021) *Persons with Disability Equity Act, 2021*. Maseru: Government of Lesotho.

Martin, R.C. (2017) *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston: Prentice Hall.

OWASP (n.d.) *Password Storage Cheat Sheet*. Available at: https://cheatsheetseries.owasp.org (Accessed: 25 October 2026).

Sommerville, I. (2016) *Software Engineering*. 10th edn. Harlow: Pearson.

The libsodium project (n.d.) *libsodium documentation: Password hashing*. Available at: https://doc.libsodium.org (Accessed: 25 October 2026).

The SQLite project (n.d.) *SQLite documentation*. Available at: https://www.sqlite.org/docs.html (Accessed: 25 October 2026).


# Section 14: Appendices

## Appendix A: Requirements traceability matrix

Each functional requirement is linked to its use case, design elements, source files and test cases. Test names are GoogleTest suite names in the `tests` folder of the repository.

| Req | Use case | Classes and methods | Tables | Source files | Tests |
|---|---|---|---|---|---|

| Req | Use case | Classes and methods | Tables | Source files | Tests |
|---|---|---|---|---|---|
| FR-01 | Log in | `AuthService::login`, `IPasswordHasher` | User | `AuthService.cpp`, `SodiumPasswordHasher.cpp` | `Auth.*`, `PasswordHasher.*`, `SystemTest.*` |
| FR-02 | All | `User::can`, `AccessControl`, four role classes | User | `User.cpp`, `AccessControl.cpp` | `Access.*`, `SystemTest.AccessIsLimitedByRole...` |
| FR-03 | Manage users | `UserService` | User | `UserService.cpp` | `Admin.CreateUser...`, `Admin.Deactivate...` |
| FR-04 | Manage academic data | `AcademicService` | Department, Course, CourseLecturer, Enrolment | `AcademicService.cpp` | `Admin.*`, `SystemTest.AdministratorSetsUp...` |
| FR-05 | Create assignment | `AssignmentService::createAssignment` | Assignment | `AssignmentService.cpp` | `Assignments.*` |
| FR-06 | View assignments | `AssignmentService::listForStudent` | Assignment, Enrolment | `AssignmentService.cpp` | `Assignments.StudentSees...` |
| FR-07 | Submit assignment | `SubmissionService::submit` | Submission | `SubmissionService.cpp` | `Submit.*`, `SubmitBoundary.*` |
| FR-08 | Resubmit | `SubmissionService::submit` | Submission | `SubmissionService.cpp` | `Submit.Resubmission...` |
| FR-09 | Grant extension | `AssignmentService::grantExtension`, `DeadlineResolver` | Extension | `AssignmentService.cpp`, `DeadlineResolver.h` | `Extensions.*`, `SystemTest.ExtensionFlow...` |
| FR-10 | Submit, Mark | `ILatePenaltyStrategy`, `StandardLatePenalty` | none | `LatePenalty.cpp` | `LateDaysBoundary.*`, `PenaltyMarks.*`, `LatePenalty.*`, `Marking.LatePenalty...` |

| Req | Use case | Classes and methods | Tables | Source files | Tests |
|---|---|---|---|---|---|
| FR-11 | Mark submission | `AssessmentService::recordMark` | Submission | `AssessmentService.cpp` | `Marking.*` |
| FR-12 | Review and release marks | `AssessmentService::moderate`, `release`, `resultsForStudent` | Moderation, Submission | `AssessmentService.cpp` | `Moderation.*`, `SystemTest.EndToEnd...` |
| FR-13 | Mark, view history | `AssessmentService::recordMark`, `auditTrail` | MarkAudit | `AssessmentService.cpp`, `SqliteRepositories.cpp` | `Marking.ChangingAnExistingMark...`, `Marking.MarkAndAuditAreWrittenInOneTransaction` |
| FR-14 | All | `listCourses`, `submissionsForAssignment`, `awaitingModeration` | none | `AcademicService.cpp`, `AssessmentService.cpp` | `Admin.CourseListings...`, `Marking.OnlyTheCourseLecturer...` |

## Appendix B: Risk register, original and updated

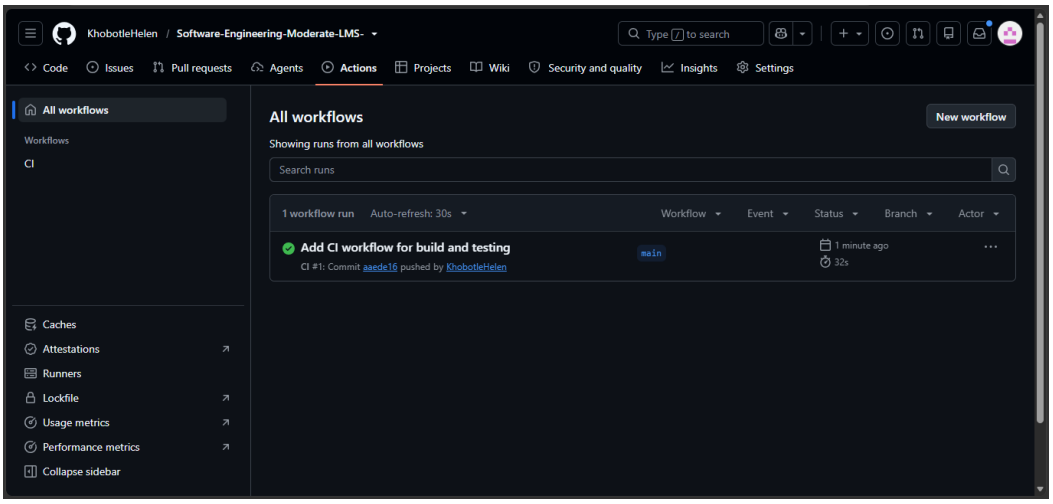
Likelihood and impact are scored 1 (very low) to 5 (very high); exposure is likelihood times impact. The register was updated after implementation and testing.

| ID | Risk | L | I | Exposure | Owner | Status after implementation |
|---|---|---|---|---|---|---|
| 1 | Old school computers may not run the application adequately | 4 | 4 | 16 | Developer | Reduced: all list queries and login are well under 2 s with 800 students on the development machine; still to be repeated on the school computer (exposure now 8) |
| 2 | Unreliable network affects operation | 4 | 4 | 16 | Developer | Closed: the system uses a local SQLite file and contains no network code |
| 3 | Requirement conflicts cause | 4 | 4 | 16 | Project manager | Closed: conflicts resolved before the SRS was baselined; two open rules (part-day lateness, |

| ID | Risk | L | I | Exposure | Owner | Status after implementation |
|---|---|---|---|---|---|---|
| | rework | | | | | resubmission) were decided during implementation and documented |
| 4 | Users struggle to use the system | 3 | 4 | 12 | Project manager | Open: menus are simple and role-specific, but the usability test with one user per role (NFR-07) has not been run |
| 5 | Stakeholders disagree about late-penalty or extension rules | 3 | 4 | 12 | Project manager | Reduced: rules are implemented in one replaceable Strategy class and documented; school should confirm them (exposure now 6) |
| 6 | Insufficient testing time leaves defects unresolved | 3 | 5 | 15 | Developer | Reduced: 112 automated tests run on every push; four defects found and fixed (exposure now 5) |

## Appendix C: Test and quality evidence

| Evidence | Result | Repository file |
|---|---|---|
| Test results | 112 tests, 19 suites, all passed | `docs/qa/test_results.txt` |
| Coverage, business-logic layer | 98.3% (711 of 723 lines) | `docs/qa/coverage_business_logic.txt` |
| Coverage, all of `src` except `main.cpp` | 92.5% (1,550 of 1,676 lines) | `docs/qa/coverage_all_src.txt` |
| Static analysis | 12 findings fixed, 0 remaining | `docs/qa/cppcheck_before.txt`, `docs/qa/cppcheck_after.txt` |
| Code metrics | 2,887 lines, 236 functions, average complexity 2.5 | `docs/qa/metrics_lizard.txt` |
| Continuous integration | Workflow `CI` run #1 on `main` passed in 32 s | GitHub Actions tab; screenshot below |




## Appendix D: Documents in the repository

| File | Purpose |
|---|---|
| `README.md` | Overview and folder map |
| `docs/BUILD.md` | Build, run, test and analysis instructions |
| `docs/CODING_STANDARD.md` | Documented coding standard |
| `docs/GIT_WORKFLOW.md` | Branching strategy and release tagging |
| `docs/DESIGN_DEVIATIONS.md` | Differences between the Phase 3 design and the implementation, and known limitations |
| `docs/TRACEABILITY.md` | Full traceability table |
| `.github/workflows/ci.yml` | Continuous-integration pipeline |

## Appendix E: User manuals

The menu options available to each role are listed in Section 9.3. For each role, the manual should give the steps for the main tasks: students log in, open My assignments, choose Submit or resubmit work, and later check My released marks; lecturers create assignments, grant extensions, mark submissions and check the mark history; Heads of Department open the marks waiting for review, approve or return each one, and release approved marks; administrators create accounts, departments, courses, assignments of lecturers and enrolments. Dates are typed as `YYYY-MM-DD HH:MM`, a number chooses a menu option, and 0 logs out.